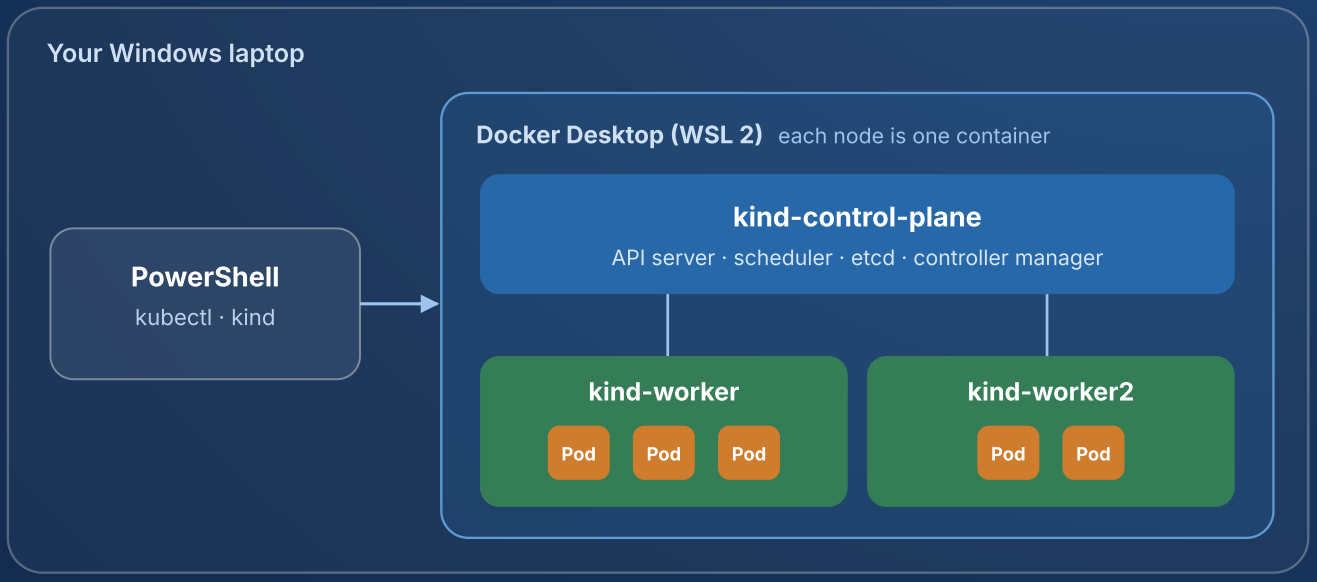
STUDENT LAB MANUAL · STEPS 1–6

Kubernetes Lab on Windows

Build a three-node Kubernetes cluster on your own laptop, deploy an NGINX web app, then scale it and watch Kubernetes heal itself.



1 **Build**
a cluster with 1 control plane and 2 workers

2 **Deploy**
three NGINX Pods and see where they run

3 **Scale and heal**
go to six Pods and watch one come back

STEPS	TIME	SYSTEM	LEVEL
6 parts, 27 steps	About 60 min	Windows 10 / 11	Beginner

WSL 2 · Docker Desktop · kubectl · kind v0.33 · NGINX

Before you begin

Goal: build a local Kubernetes cluster with one control-plane node and two worker nodes, deploy NGINX, and test scaling and self-healing.

Check your laptop

- [ ] Windows 10 or 11, 64-bit
- [ ] Internet access (about 2 GB to download)
- [ ] Administrator permission to install software
- [ ] 8 GB RAM at least (16 GB recommended)
- [ ] PowerShell (not Git Bash)

How to use this manual

Work through the steps **in order**. Copy one command at a time into PowerShell and press **Enter**.

- Read **What it does** before you run it.
- Compare your screen with **You should see** before moving on.
- Replace anything in `<angle brackets>` with a real name, without the brackets.
- Tick each step's box when it's done.

The lab at a glance

1 Install the tools
WSL 2, Docker Desktop, kubectl, kind
20 min

2 Check the tools
Make sure each one runs
5 min

3 Write the config file
Describe a 3-node cluster
5 min

4 Build the cluster
Create it and check it is Ready
10 min

5 Deploy NGINX
Run 3 Pods and see where they land
10 min

6 Scale and self-heal
Go to 6 Pods, delete one, watch it return
10 min

How to read a step

2.4 Check kind ☐

PowerShell

```
kind version
```

What it does Shows the installed kind version.

You should see A version line, such as `kind v0.33.0`.

1. **Step number and name.** Tick the box when the step is done.
2. **The command.** Type or copy it into PowerShell (the arrow is just a marker), then press Enter.
3. **What it does**, in one line.
4. **You should see:** check this before moving on.

PART 1 OF 6

01 Install the tools

about 20 min

1 2 3 4 5 6

You need four tools: **WSL 2** (the Linux layer), **Docker Desktop** (runs the nodes), **kubectl** (talks to the cluster) and **kind** (creates the cluster).

1.1 Install WSL 2

PowerShell — Run as Administrator

```
wsl --install
```

What it does Installs Windows Subsystem for Linux (WSL 2), which Docker Desktop runs on.

You should see Download and install progress, then a request to restart. **Restart Windows** when asked.

1.2 Install Docker Desktop

DO THIS

1. Download **Docker Desktop for Windows** from docker.com/products/docker-desktop.
2. Run the installer and keep **"Use WSL 2 instead of Hyper-V"** ticked.
3. Open **Docker Desktop** from the Start menu and accept the terms.

What it does Installs Docker, which will run each Kubernetes node as a container. This step uses the installer, not a PowerShell command.

You should see Docker Desktop's status shows **Engine running**.

Important: If Docker Desktop says WSL 2 or virtualization is missing, finish the WSL setup and restart Windows. If needed, turn on CPU virtualization (Intel VT-x or AMD-V/SVM) in your BIOS/UEFI settings.

1.3 Install kubectl

PowerShell

```
winget install -e --id Kubernetes.kubectl
```

What it does Installs **kubectl**, the command-line tool used to control Kubernetes.

You should see `Successfully installed` at the end.

First time using winget? It may ask you to accept the source agreements. Type `Y` and press Enter.

1.4 Install kind

PowerShell

```
winget install -e --id Kubernetes.kind
```

What it does Installs **kind** (Kubernetes IN Docker), which creates a Kubernetes cluster using Docker containers.

You should see `Successfully installed` at the end.

1.5 Reopen PowerShell

PowerShell

```
exit
```

What it does Closes this PowerShell window. Open a **new** one so it can find the tools you just installed.

PART 2 OF 6

02 Check that the tools work

about 5 min

1 2 3 4 5 6

Open **Docker Desktop** and wait until the Engine is running. Then, in a new PowerShell window, run these commands one by one.

2.1 Check Docker

PowerShell

```
docker version
```

What it does Shows the versions of Docker's client and server.

You should see Both a **Client** section and a **Server** section. If the Server part shows an error, Docker Desktop is not running yet.

2.2 Test a container

PowerShell

```
docker run hello-world
```

What it does Downloads and runs a tiny test container.

You should see Sample output (yours will look similar)

```
Hello from Docker!
This message shows that your installation appears to be working correctly.
```

2.3 Check kubectl

PowerShell

```
kubectl version --client
```

What it does Shows the installed kubectl version.

You should see Sample output (yours will look similar)

```
Client Version: v1.37.x
Kustomize Version: v5.x.x
```

2.4 Check kind

PowerShell

```
kind version
```

What it does Shows the installed kind version.

You should see Sample output (yours will look similar)

```
kind v0.33.0 go1.26.x windows/amd64
```

PART 3 OF 6

03 Create the cluster configuration file

about 5 min

1 2 3 4 5 6

A short file tells kind to build **one control-plane node and two worker nodes** instead of a single node.

3.1 Create a lab folder

PowerShell

```
mkdir k8s-lab
```

What it does Creates a folder to keep the lab files in. If it already exists, just continue.

3.2 Enter the folder

PowerShell

```
cd k8s-lab
```

What it does Moves PowerShell into the lab folder. Run all the following commands from here.

You should see The prompt now ends in `\k8s-lab>`.

3.3 Open the configuration file

PowerShell

```
notepad kind-config.yaml
```

What it does Opens Notepad with a new file called kind-config.yaml.

You should see A question asking whether to create a new file. Click **Yes**.

3.4 Paste the configuration and save

DO THIS

1. Copy the content below into Notepad **exactly as shown**.
2. Press **Ctrl + S** to save, then close Notepad.

What it does Asks kind to create three nodes: one control-plane node and two worker nodes.

kind-config.yaml

```
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
  - role: control-plane
  - role: worker
  - role: worker
```

Use spaces, not tabs. If Notepad offers a *Save as type*, choose **All files** so the file does not become kind-config.yaml.txt.

3.5 Check that the file exists

PowerShell

```
Get-ChildItem
```

What it does Lists the files in the current folder.

You should see **kind-config.yaml** in the Name column.

PART 4 OF 6

04 Build the three-node Kubernetes cluster

about 10 min

1 2 3 4 5 6

One command builds the whole cluster. Then you check that all three nodes are healthy and look at what is running inside.

4.1 Create the cluster

PowerShell

```
kind create cluster --config kind-config.yaml
```

What it does Creates the cluster. The first run can take a few minutes while the Kubernetes node image downloads.

You should see

Sample output: the last line is the one that matters

```
Creating cluster "kind" ...
 ✓ Ensuring node image (kindest/node:v1.37.0)
 ✓ Preparing nodes
 ✓ Writing configuration
 ✓ Starting control-plane
 ✓ Installing CNI
 ✓ Installing StorageClass
 ✓ Joining worker nodes
Set kubectl context to "kind-kind"
```

4.2 Check the cluster connection

PowerShell

```
kubectl cluster-info --context kind-kind
```

What it does Checks that kubectl can talk to the new cluster.

You should see

Sample output (yours will look similar)

```
Kubernetes control plane is running at https://127.0.0.1:<port>
CoreDNS is running at https://127.0.0.1:<port>/api/v1/...
```

4.3 Check node status

PowerShell

```
kubectl get nodes
```

What it does Lists the control-plane node and the two worker nodes.

You should see Three nodes with STATUS **Ready**. If one is still starting, wait a minute and run it again.

Sample output (yours will look similar)

```
NAME                 STATUS   ROLES           AGE   VERSION
kind-control-plane   Ready    control-plane   2m    v1.37.0
kind-worker          Ready    <none>          90s   v1.37.0
kind-worker2         Ready    <none>          90s   v1.37.0
```

4.4 Check the Docker containers

PowerShell

```
docker ps
```

What it does Shows the running Docker containers. Each one is a kind node.

You should see Three containers named `kind-control-plane`, `kind-worker` and `kind-worker2` in the NAMES column.

4.5 Check the Kubernetes system Pods

PowerShell

```
kubectl get pods -n kube-system -o wide
```

What it does Lists Kubernetes' own components, such as the API server and the scheduler.

You should see Pods such as `etcd`, `kube-apiserver`, `kube-scheduler` and `kube-controller-manager` on **kind-control-plane**, and a `kube-proxy` Pod on every node. Some may need a little time to show **Running**.

What just happened? The control-plane node is the cluster's brain: it stores the state and decides where work goes. The two workers are the muscles where your apps will run. All three are Docker containers on your laptop.

PART 5 OF 6

05 Deploy an NGINX web application

about 10 min

1 2 3 4 5 6

Now ask Kubernetes to run a real app: three copies of the NGINX web server, spread over the workers.

5.1 Create three NGINX replicas

PowerShell

```
kubectl create deployment web --image=nginx --replicas=3
```

What it does Creates a Deployment named **web** and asks Kubernetes to run three copies (Pods) of the NGINX web server.

You should see Sample output (yours will look similar)

```
deployment.apps/web created
```

5.2 Check the Deployment

PowerShell

```
kubectl get deployments
```

What it does Shows how many replicas you asked for and how many are ready.

You should see Wait until READY shows **3/3**.

Sample output (yours will look similar)

```
NAME   READY   UP-TO-DATE   AVAILABLE   AGE
web    3/3     3            3           45s
```

5.3 See where the Pods are running

PowerShell

```
kubectl get pods -o wide
```

What it does Lists the three Pods and the node each one was placed on.

You should see Look at the **NODE** column: the Pods are spread over `kind-worker` and `kind-worker2`.

Sample output, some columns left out (your Pod names will differ)

```
NAME                   READY   STATUS    AGE   IP           NODE
web-6c8d9f7b54-2xk8p   1/1     Running   50s   10.244.1.2   kind-worker
web-6c8d9f7b54-q7wzm   1/1     Running   50s   10.244.2.2   kind-worker2
web-6c8d9f7b54-t9s4d   1/1     Running   50s   10.244.1.3   kind-worker
```

Record: Pods on kind-worker _____ Pods on kind-worker2 _____

5.4 Inspect a Pod

OPTIONAL

PowerShell

```
kubectl describe pod <pod-name>
```

What it does Shows the details and history of one Pod. Replace `<pod-name>` with a real name copied from Step 5.3, without the angle brackets.

You should see Under **Events** at the bottom, a line like `Successfully assigned default/web-… to kind-worker`. That is the Scheduler's decision.

Copying a Pod name: select it with the mouse, press **Ctrl + C** (or right-click), then paste it into your command with **Ctrl + V**.

PART 6 OF 6

06 Scale the Pods and test self-healing

Change how many copies you want, then delete one on purpose and watch Kubernetes put it back without being asked.

about 10 min

1 2 3 4 5 6

6.1 Scale from three Pods to six

PowerShell

```
kubectl scale deployment web --replicas=6
```

What it does Changes the desired number of NGINX Pods from three to six.

You should see Sample output (yours will look similar)

```
deployment.apps/web scaled
```

6.2 Check the six Pods

PowerShell

```
kubectl get pods -o wide
```

What it does Shows all the Pods and the node each one was placed on.

You should see Six `web` Pods with STATUS **Running**, shared between the two workers.

Record: Pods on kind-worker _____ Pods on kind-worker2 _____

Watch it happen live: run `kubectl get pods -w` in a second PowerShell window before Step 6.3. Press **Ctrl + C** to stop watching.

6.3 Delete one Pod for the test

PowerShell

```
kubectl delete pod <pod-name>
```

What it does Deletes one Pod on purpose. Replace `<pod-name>` with a real web Pod name from Step 6.2.

You should see A line confirming that the Pod was deleted.

6.4 Confirm self-healing

PowerShell

```
kubectl get pods
```

What it does Checks that Kubernetes created a replacement Pod to bring the Deployment back to six replicas.

You should see Six Pods again. The replacement has a **new name** and an AGE of only a few seconds.

What you have learned

You installed the tools, created a three-node local cluster, deployed NGINX, scaled the application and saw Kubernetes replace a deleted Pod automatically.

Installed the tools
WSL 2, Docker Desktop, kubectl and kind on Windows.

Built a cluster
One control-plane node and two workers, all running as Docker containers.

Looked inside
Saw the API server, scheduler, etcd and kube-proxy running as Pods.

Deployed an app
Ran three NGINX Pods and saw which node each one was placed on.

Scaled and healed
Went from 3 to 6 Pods and watched a deleted Pod come back by itself.

If something goes wrong

What you see	What to do
`Cannot connect to the Docker daemon` or `error during connect`	Docker Desktop isn't running. Open it and wait for **Engine running**.
`kubectl` or `kind` is *not recognized*	Close every PowerShell window and open a new one (Step 1.5).
WSL 2 or virtualization missing	Run `wsl --install` as Administrator and restart. Turn on virtualization in BIOS/UEFI.
A node stays **NotReady**	Wait 1–2 minutes. Still stuck? Close other apps, run `kind delete cluster`, then repeat Step 4.1.
A Pod shows **ImagePullBackOff**	No internet or a typo in the image name. `kubectl describe pod <pod-name>` shows the exact error.
error parsing kind-config.yaml	The file has tabs or wrong spacing. Re-copy it exactly as shown in Step 3.4.
Commands hang or behave oddly in Git Bash	Use PowerShell instead.

My notes

Command quick reference

Every command from Steps 1–6, in the order you used them.

Part	Command	What it does
1	`wsl --install`	Install WSL 2 (as Administrator)
1	`winget install -e --id Kubernetes.kubectl`	Install kubectl
1	`winget install -e --id Kubernetes.kind`	Install kind
2	`docker version`	Check Docker client and server
2	`docker run hello-world`	Test that containers run
2	`kubectl version --client`	Check kubectl
2	`kind version`	Check kind
3	`mkdir k8s-lab`	Make the lab folder
3	`cd k8s-lab`	Go into the lab folder
3	`notepad kind-config.yaml`	Create the config file
3	`Get-ChildItem`	List files in the folder
4	`kind create cluster --config kind-config.yaml`	Build the 3-node cluster
4	`kubectl cluster-info --context kind-kind`	Check the connection
4	`kubectl get nodes`	List nodes and their status
4	`docker ps`	See the node containers
4	`kubectl get pods -n kube-system -o wide`	See the system Pods
5	`kubectl create deployment web --image=nginx --replicas=3`	Run 3 NGINX Pods
5	`kubectl get deployments`	Check desired vs ready replicas
5	`kubectl get pods -o wide`	See Pods and their nodes
5	`kubectl describe pod <pod-name>`	Details and events of one Pod
6	`kubectl scale deployment web --replicas=6`	Scale to 6 Pods
6	`kubectl delete pod <pod-name>`	Delete one Pod
6	`kubectl get pods`	Check the replacement Pod